Applied AI Engineer

Take-Home Assessment

Build a small full-stack agent and compare two ways of doing retrieval: Naive RAG and StreamRAG.

At a glance

Time budget: 3 days. Friday, Saturday, and Sunday.

Deliverables: A public GitHub repo, a video of 5 minutes or less, and a short benchmark write-up (md, pdf, or docx).

Tools: Use any AI tool you like (Claude, Cursor, Codex, and so on). We expect it.

What we grade: How well you understand and can own what you build, not the amount of code.

1. What this is

As an Applied AI Engineer here, you will own projects from the first idea through to something running in production and being maintained. This task is a small version of that loop. It should take a focused weekend, not a marathon.

You will build a **small full-stack application** that runs a general-purpose AI agent, and you will compare two ways of feeding it information:

- **Path A, Naive RAG:** the standard approach. Data goes into vectors, you search those vectors for the closest matches, you put the matches into the prompt as context, and the model answers.
- **Path B, StreamRAG:** the agent applies the StreamRAG idea, which is to start retrieval in parallel with the incoming query instead of strictly waiting for it to finish. The goal is lower latency and better factual grounding.

You then **compare the two paths on speed, cost, accuracy, and overall behaviour**, and write up what you found.

> **What we are really looking for**
>
> We want to understand the person behind the submission. We are not scoring the code line by line.
>
> Using AI coding tools is part of the job here, so use them freely. What matters is that you understand what you shipped, can explain the trade-offs, and could keep building on it in a real setting.
>
> It is completely fine if you do not finish everything. A smaller piece you fully understand and can defend is worth more than a large one you cannot.

---

2. Scope, and keeping it small

Build the smallest thing that lets you honestly compare the two paths. Please do not over-build. A clean, working, well-understood minimum is exactly the target.

The agent and its harness

Build a general-purpose agent with a basic but real harness. Each piece can be simple and honest. Please cover:

- **Tools:** at least one working tool the agent can call, such as a web search, a calculator, or a data lookup.
- **Memory:** some form of memory that carries across turns of a conversation.
- **Context management and compression:** show how you keep the context within budget, for example by summarising, trimming, or selecting what to keep.
- **Sub-agent or skills:** at least one of these. Either a delegated sub-agent, or a reusable skill the agent can invoke. One is enough. Both are nice-to-have, not a requirement.

The two retrieval paths

- **Path A, Naive RAG:** a simple retrieve-then-answer pipeline over a small set of documents you pick.
- **Path B, StreamRAG:** apply the StreamRAG concept. If you build a spoken or streaming interface, retrieval should begin before the input is complete. If you stay text-only, you can approximate this by kicking off retrieval in parallel with generation and streaming results in. Say clearly in your write-up which version you chose and why.

The full-stack app

- **A minimal frontend:** one page is fine. A user should be able to send a query and see the answer from either path.
- **A backend:** serves the agent and both retrieval paths, and records the numbers below.
- **Deployment [Optional]:** deploy it somewhere we can reach on any free tier, or give us a one-command local run. If you run out of time to deploy, a clean local setup is fine. Just tell us.

3. Suggested stack

Use whatever you are fastest and most confident in. Here is a guide that fits how we work, so you are free to follow it or pick equivalents:

Layer	Options you can use
Backend / agent	Python with FastAPI, plus a framework of your choice: LangChain, Agno, Pydantic AI, or similar.
Frontend	Next.js, or any modern framework you prefer.
Packaging	Docker, so the app is easy to run and deploy.
LLM provider	Any of Groq, Anthropic, Google, or OpenAI. Free or low-cost tiers are fine.

On low-level code: if you are comfortable with a language like C, C++, or Rust, we would love to see one meaningful piece where it earns its place. For example a fast similarity or embedding routine, a text-processing step, or a latency-sensitive part of the StreamRAG path called from your backend. This is a plus, not a hard requirement. If you use it, explain in your video why you chose it and what you gained.

---

4. What to measure

Run both paths over the same set of queries and report these four things:

Dimension	What it means	A simple metric
Speed	How fast the user gets a useful answer	Total response time, and time to first token
Cost	What each query costs to run	Tokens used and rough cost per query, plus number of calls
Accuracy	How correct and well-grounded the answer is	A score against a small set of expected answers you define
Performance	How it behaves across the whole test set	Throughput, any failures, and your own summary judgement

Your test set: a small, fixed set of queries with expected answers you write up front. Ten to twenty is plenty. Run both paths over the same set so the comparison is fair, and make it easy for us to re-run from your repo.

> **Honest results beat impressive ones**

> A correct, clearly explained comparison on a small set is worth more to us than a flashy chart you cannot defend.
>
> If StreamRAG does not win on every measure, say so and explain why. Understanding the trade-off is the real deliverable.

5. What to hand in

1. A public GitHub repository

- All the source code, runnable from a clean checkout.
- **A README that covers:** what it does, how to run it, the architecture at a glance, and any decisions or shortcuts you made.
- The benchmark, with scripts or clear steps so we can reproduce your numbers.

2. A video of 5 minutes or less

- Start with a short introduction of yourself.
- Explain the project and the main decisions you made.
- Walk us through it running, and show the two paths being compared.
- **Record your screen and your voice.** This is how we get to know you, so talk us through your own thinking, not just the code.

3. A short benchmark write-up

- A `.md`, `.pdf`, or `.docx` file.
- The four measures for both paths, a description of your test set, the results, and a short conclusion on when each approach is the better choice.

> **Quick checklist before you submit**
> Public GitHub repo link, with code, README, and benchmark steps
> Video of 5 minutes or less, covering intro, explanation, and a walkthrough
> Benchmark write-up (md, pdf, or docx) included in the Repo
> The repo runs from a clean checkout, or the app is deployed and reachable

---

6. A suggested pace

Three days. This is just one way to shape it. Adjust as you see fit:

Day	Focus
Friday	Decide your scope, pick your documents and test queries, and get the agent plus the Naive RAG path working.
Saturday	Add the StreamRAG path and any low-level piece, wire up the frontend, and get both paths answered.
Sunday	Run the benchmark, write the report, record the video, deploy or finalise the run steps, and submit.

---

7. How we evaluate

We are assessing you as an engineer, not counting features. Roughly:

What we look for	Weight
Understanding and ownership. Can you explain every choice and defend the trade-offs	35%
A correct grasp of the ideas: RAG, StreamRAG, the agent harness, and benchmarking	25%
A working full-stack build, from idea to running app, however small	20%
Sensible use of a low-level language where it adds value	10%
How clear your video and write-up are	10%

A few notes for you

Use any AI tool you want. We assume you will. Just make sure you understand and can defend what ships.

Partial completion is fine. Be upfront in the video and README about what is done, what is stubbed out, and what you would do next.

Keep it small and honest. A tight, well-understood submission is exactly what we are hoping to see.

Good luck. We are looking forward to seeing how you think.